

Knowledge-Gap Action Decisions for LLM QA

An experiment on answer / ask / retrieve / abstain / challenge decisions

Dayi Yao 2022011550

Tsinghua University

June 10, 2026

Talk Plan

- ▶ Problem: why action selection matters before generation
- ▶ Project design: data schema, labels, and evaluation targets
- ▶ Engineering implementation: code structure and main functions
- ▶ Results: method comparison, ablations, stability, and errors
- ▶ Limitations: current evidence, boundaries, and next validation targets

Problem: Answering Is Only One Possible Action

- ▶ Many user questions require a decision before final answering.
- ▶ Missing user constraints call for **asking a question**.
- ▶ Missing or fresh facts call for **retrieval**.
- ▶ Unsupported high-stakes claims call for **abstention**.
- ▶ Contradicted assumptions call for **challenging the premise**.

The project treats this as a supervised action-decision task.

Task Definition

Inputs

- ▶ user query
- ▶ dialogue context
- ▶ retrieved evidence
- ▶ candidate answer

Outputs

- ▶ gap_type: explanation label
- ▶ gold_action: operational label
- ▶ action set: answer, ask, retrieve, abstain, challenge

Evaluation Target

- ▶ Primary classification metric: action macro-F1.
- ▶ Safety-sensitive metric: wrong-answer rate.
- ▶ Utility combines correctness, wrong-answer penalty, over-refusal, over-asking, and turn cost.
- ▶ The experiment evaluates the next action before any final long answer.

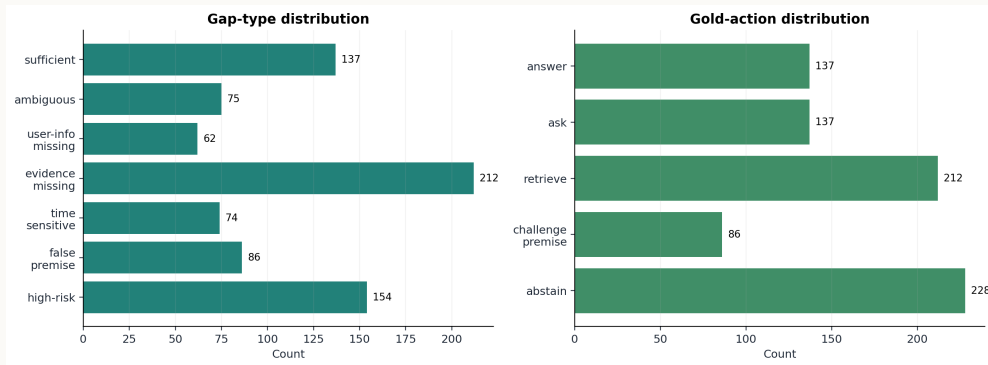
Related Work Used as Background

- ▶ AmbigQA and ASQA motivate ambiguous-question and clarification cases.
- ▶ FreshLLMs motivates time-sensitive retrieval decisions.
- ▶ SelfCheckGPT motivates consistency-style uncertainty features.
- ▶ Self-RAG motivates retrieval as a controllable action.
- ▶ Sufficient Context and abstention benchmarks motivate evidence-sufficiency and refusal boundaries.

Dataset Overview

- ▶ Full run: 800 records generated under controlled knowledge-gap scenarios.
- ▶ Split: train 483, validation 156, test 161.
- ▶ Split is by `group_id`, keeping paired variants in the same split.
- ▶ DeepSeek generation is cached; self-consistency feature generation requires a live API key.

Label Distribution



Record Schema

- ▶ `schema.py` defines `GapType`, `Action`, required fields, and validation.
- ▶ Key text fields: `user_initial_query`, `dialogue_context`, `retrieved_evidence`, `candidate_answer`.
- ▶ Key supervision fields: `gap_type`, `gold_action`, `required_slots`.
- ▶ Annotation fields such as risk and evidence sufficiency are recorded for labels and evaluation.

Controlled Gap Types

- ▶ `sufficient_information`: answer is justified.
- ▶ `ambiguous_question` and `user_info_missing`: ask before answering.
- ▶ `evidence_missing` and `time_sensitive`: retrieve or abstain depending on the claim.
- ▶ `false_premise`: correct the assumption first.
- ▶ `high_risk_or_expert_needed`: avoid unsupported expert advice.

Repository Structure

- ▶ `knowledge_gap_decision/`: core package.
- ▶ `data/processed/`: generated dataset, splits, and feature CSVs.
- ▶ `results/`: metrics, predictions, ablations, significance tests, and figures.
- ▶ `reports/`: generated report and experiment log.
- ▶ `tests/`: schema, split-isolation, feature, and metric checks.

Data Construction Code

- ▶ `build_deepseek_dataset`: calls the JSON-mode DeepSeek client and normalizes generated records.
- ▶ `build_dataset`: deterministic fallback dataset generator.
- ▶ `validate_dataset`: catches missing fields, invalid labels, and malformed lists.
- ▶ `split_by_group`: keeps related records inside one train / val / test split.
- ▶ `write_dataset`: writes JSONL splits and label distribution artifacts.

Feature Extraction Code

- ▶ `compute_features`: central feature table builder.
- ▶ Question-side features: length, time words, condition words, subjective words, entity count, risk keywords.
- ▶ Evidence-side features: TF-IDF top similarity, token overlap, coverage ratio, contradiction proxy.
- ▶ Model-side features: DeepSeek action-vote entropy, majority ratio, rationale overlap, and vote counts.
- ▶ `feature_columns`: toggles evidence and uncertainty features for ablations.

Model Code

- ▶ `DualClassifier`: trains separate classifiers for `gap_type` and `gold_action`.
- ▶ Implemented model kinds: logistic regression, random forest, and GBDT.
- ▶ `TextClassifier`: TF-IDF text-only baseline over query, context, evidence, and candidate answer.
- ▶ `prompted_llm_baseline`: asks DeepSeek for strict JSON action predictions.
- ▶ `full_method_predict`: random-forest prediction plus conservative safety overrides.

Experiment Orchestration

- ▶ `run_experiment.run`: one entry point for dataset creation, features, models, metrics, and plots.
- ▶ `_select_full_config`: picks a Full Method variant on validation data.
- ▶ `_write_error_analysis`: writes sampled real mistakes with key feature values.
- ▶ `_write_seed_stability`: reruns the Full Method on three split seeds.
- ▶ `scripts/run_full_experiment.py` and `Makefile` provide command-line wrappers.

Evaluation Code

- ▶ `evaluate_prediction`: macro-F1, accuracy, retrieval F1, contradiction accuracy, and utility.
- ▶ `utility_components`: wrong-answer rate, over-refusal, over-asking, turn cost.
- ▶ `per_class_rows`: class-level precision / recall / F1 table.
- ▶ `significance_rows`: paired bootstrap and McNemar tests against Full Method.
- ▶ `report.py`: assembles markdown and Word reports from result files.

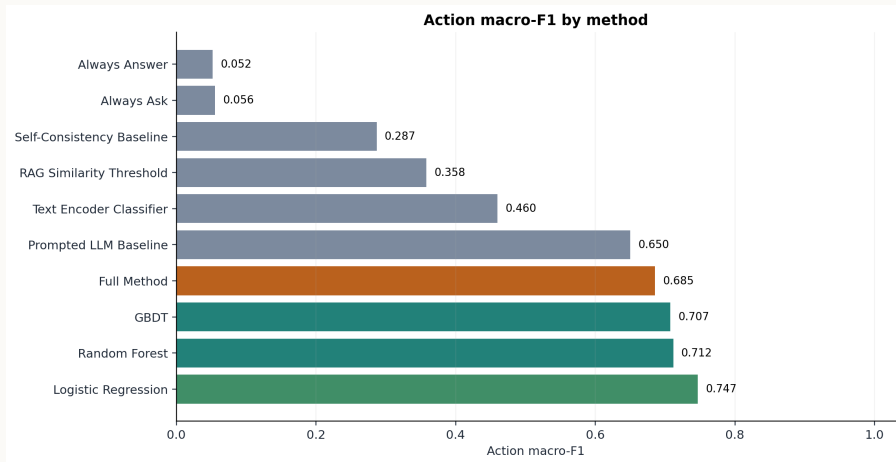
Question Ranker

- ▶ `generate_candidates`: creates clarifying questions from missing slots.
- ▶ `score_question`: combines slot coverage, uncertainty reduction, specificity, answerability, politeness, and leading-question penalty.
- ▶ `rank_questions`: writes `results/question_ranker_eval.csv`.
- ▶ The ranker improves the quality of `ask`; action-F1 mainly comes from the classifier.

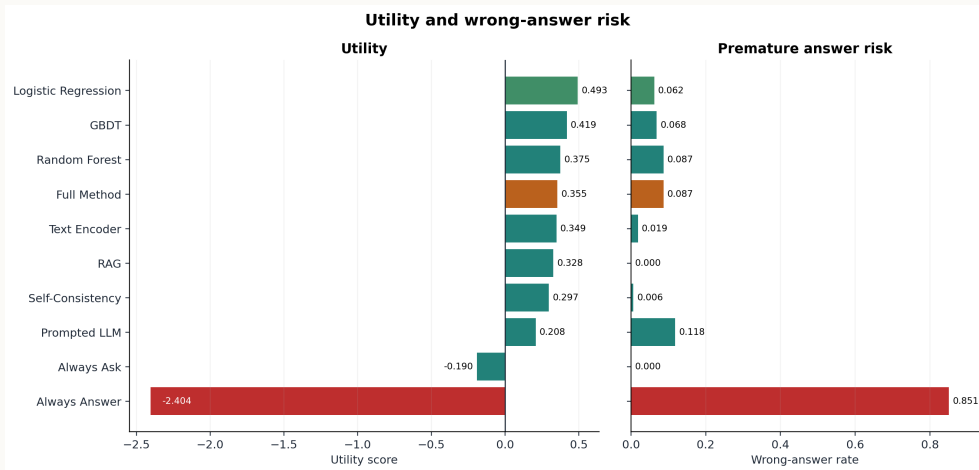
Self-consistency Feature Guardrails

- ▶ Self-consistency now samples DeepSeek five times at nonzero temperature.
- ▶ Each call receives only query, dialogue context, retrieved evidence, and candidate answer.
- ▶ Cached JSONL rows store action decisions and short rationales, never gold labels.
- ▶ Missing API keys, probe failures, or malformed LLM outputs stop the experiment.

Main Result: Action Macro-F1



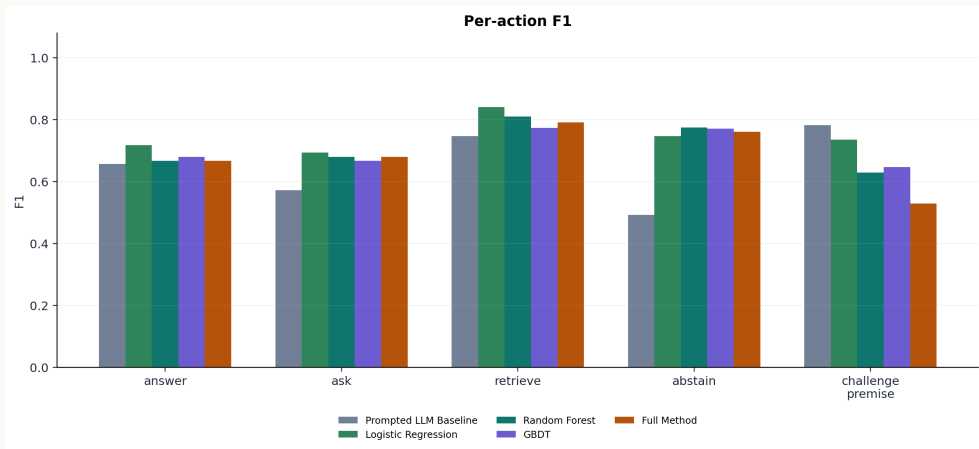
Utility and Wrong-answer Risk



Headline Numbers

- ▶ Best method in the current run: Logistic Regression, action macro-F1 0.747 and utility 0.493.
- ▶ Full Method: action macro-F1 0.685, utility 0.355, wrong-answer rate 0.087.
- ▶ Prompted LLM Baseline: action macro-F1 0.650, wrong-answer rate 0.118, and high-risk unsafe answer rate 0.176.
- ▶ Always Answer remains the failure mode: wrong-answer rate 0.851 and utility -2.404.

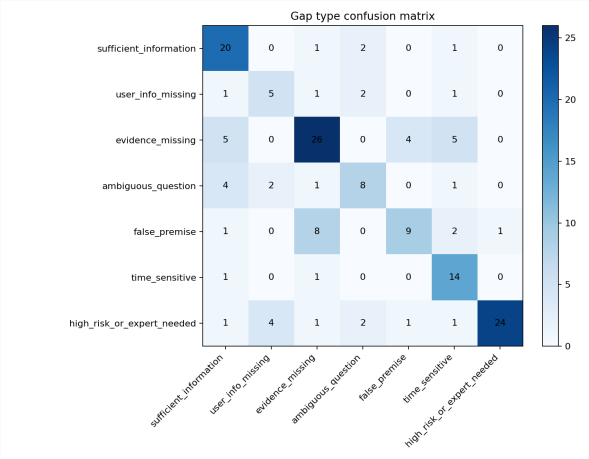
Per-action F1



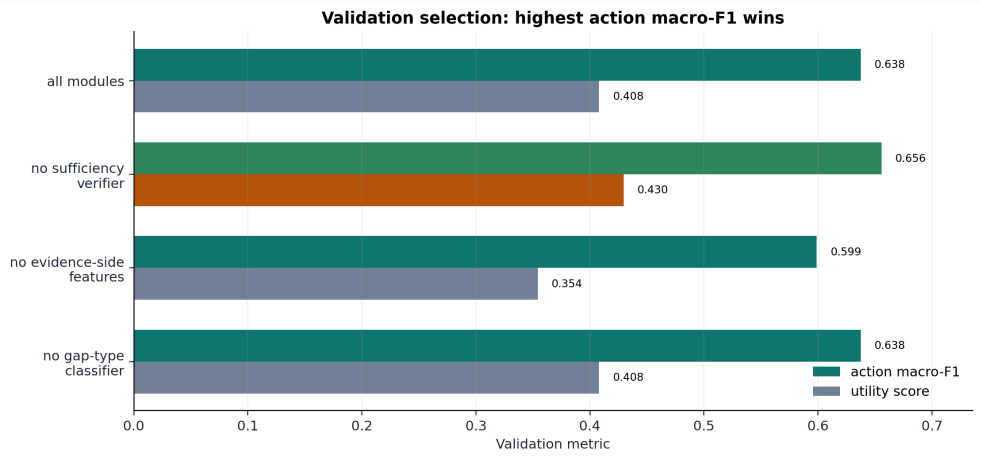
Action Confusion Matrix



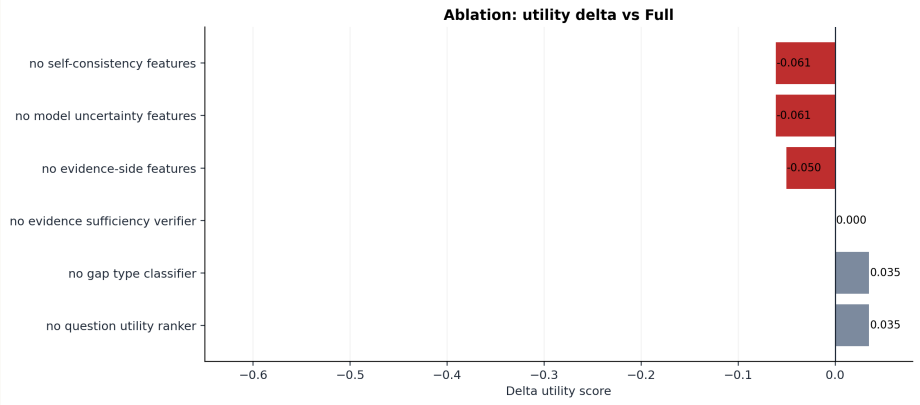
Gap-type Confusion Matrix



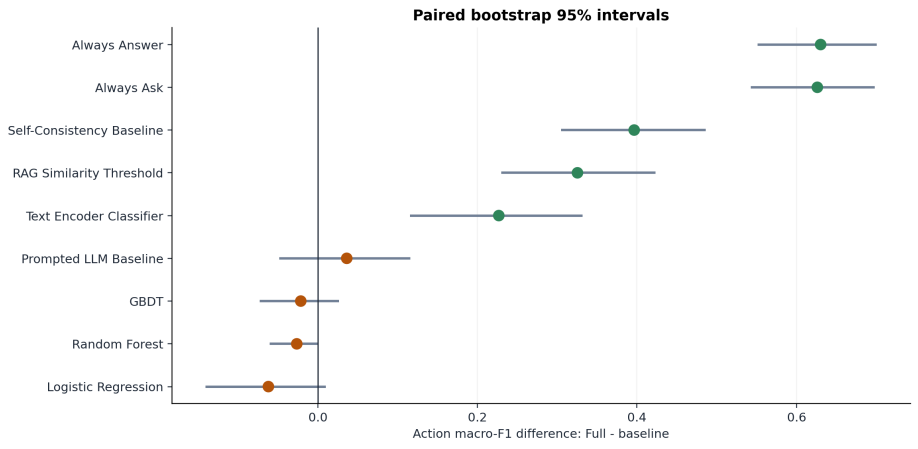
Validation Selection



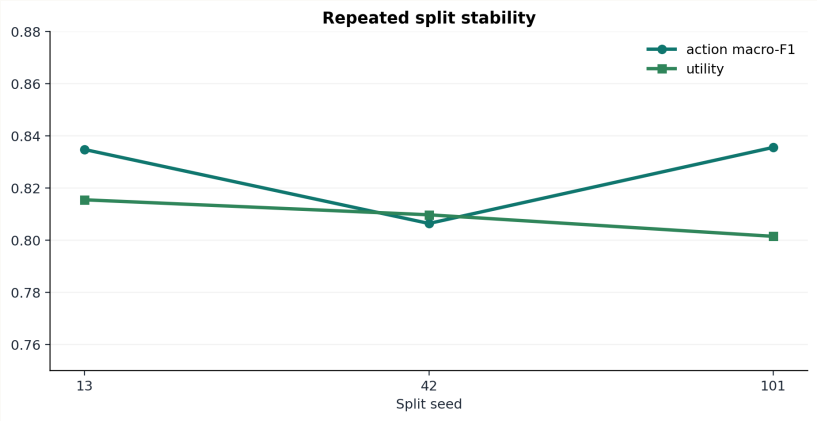
Ablation



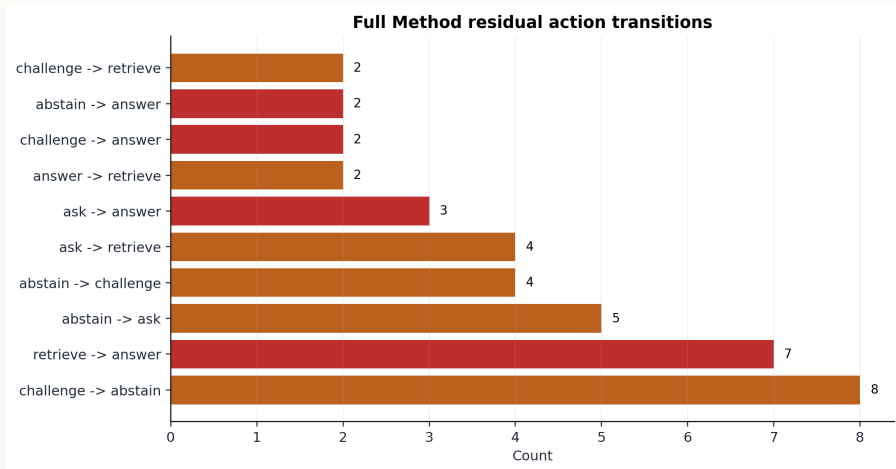
Significance Tests



Repeated Split Stability



Residual Errors



Example Boundaries

- ▶ Ambiguous technical question: ask about environment and target before giving steps.
- ▶ Current policy or deadline: retrieve fresh evidence before answering.
- ▶ False premise: state that the assumption is unsupported before continuing.
- ▶ Evidence-support judgment: abstain when supplied evidence fails to support the claim.

What the Results Say

- ▶ The current result is a realistic policy-learning problem with meaningful residual errors.
- ▶ Simple supervised learning is the strongest result: Logistic Regression beats Full Method, Random Forest, GBDT, and prompted DeepSeek on this split.
- ▶ Full Method adds engineering structure; the test split favors the simpler linear classifier.
- ▶ Prompted DeepSeek is semantically competent and answers too often in risky or under-supported cases.
- ▶ Real LLM self-consistency is useful as a feature group; direct majority voting remains weak.

Limitations

- ▶ The dataset is generated under controlled scenarios, with real-user validation still needed.
- ▶ Some labels may still reflect the generator's style and scenario design.
- ▶ The action policy is project-specific; a helpful assistant may choose ask where this policy chooses abstain.
- ▶ Evidence sufficiency is still heuristic; the selected model disables the current sufficiency verifier on validation.
- ▶ Real self-consistency depends on DeepSeek, prompt wording, temperature, and cache stability.
- ▶ The Full Method should be treated as an engineering baseline for redesign.

Next Steps

- ▶ Promote Logistic Regression to the main supervised baseline and tune thresholds directly for utility.
- ▶ Replace the heuristic sufficiency verifier with NLI, answerability, or entailment-style evidence checks.
- ▶ Add hard adjacent-label examples: challenge vs abstain, retrieve vs answer, ask vs answer, and high-risk ask vs abstain.
- ▶ Build a stronger LLM baseline with few-shot policy examples and explicit risk / evidence hierarchy.
- ▶ Validate on real user questions with human adjudication.

Takeaway

The clearest current result favors a simple, auditable action classifier.

The current evidence says that a simple, auditable action classifier can beat weak heuristics and reduce LLM over-answering, while the more complex Full Method still needs redesign.

References

- ▶ Min et al. AmbigQA: Answering Ambiguous Open-domain Questions. EMNLP 2020. [arXiv:2004.10645](#)
- ▶ Stelmakh et al. ASQA: Factoid Questions Meet Long-Form Answers. EMNLP 2022. [arXiv:2204.06092](#)
- ▶ Vu et al. FreshLLMs: Refreshing Large Language Models with Search Engine Augmentation. 2023. [arXiv:2310.03214](#)
- ▶ Manakul et al. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection. 2023. [arXiv:2303.08896](#)
- ▶ Asai et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. 2023. [arXiv:2310.11511](#)
- ▶ Sufficient Context: A New Lens on Retrieval Augmented Generation Systems. 2024. [arXiv:2411.06037](#)
- ▶ Kirichenko et al. AbstentionBench: Reasoning LLMs Fail on Unanswerable Questions. 2025. [arXiv:2506.09038](#)